

Trabajo Práctico IV - Tecnología Digital VI

Universidad Torcuato Di Tella

Benjamín Ledesma, Lucía Pizi, Milagros Siles

5 de Diciembre de 2025

Introducción

En este trabajo entrenamos y analizamos autoencoders aplicados a los audios del dataset *Speech Commands* de Google. Cada audio se transforma en un espectrograma log-Mel, que luego es comprimido por el encoder a un vector latente de dimensión reducida y reconstruido por el decoder.

El objetivo principal es estudiar cuánto puede comprimirse la señal sin perder información relevante. Para esto entrenamos modelos con distintos tamaños de vector latente ($z_{\text{dim}} = 128, 64, 32, 16, 8$), comparamos sus reconstrucciones y analizamos cómo cambia la calidad del audio y del mel generado.

Luego exploramos qué estructura aprende el modelo en el espacio latente mediante métodos como PCA, t-SNE y clustering, y evaluamos qué tan separables o agrupadas quedan las diferentes palabras.

También estudiamos la capacidad del autoencoder para realizar *denoising*, analizando cómo reconstruye un espectrograma limpio cuando la entrada se contamina con distintos niveles de ruido.

Finalmente manipulamos directamente el espacio latente mediante interpolaciones entre clases (por ejemplo *yes* → *no*) y agregando ruido al vector z , observando cómo estas transformaciones afectan el mel y el audio decodificado.

En conjunto, estos experimentos permiten entender cómo el autoencoder comprime, organiza y reconstruye la información acústica, y cuál es la dimensión latente mínima que mantiene una calidad aceptable.

1 Encodeo del audio en un vector latente

El objetivo de este inciso es construir un autoencoder capaz de transformar audios del dataset *Speech Commands* (1 segundo de duración, 16 kHz) en un vector latente de dimensión reducida, y luego reconstruirlos de manera que mantengan una calidad suficiente para reconocer su clase original. La meta es encontrar el menor tamaño posible del vector latente que conserve esta calidad de reconstrucción.

1.1 Preprocesamiento

Antes de entrenar el autoencoder, convertimos cada audio en un espectrograma log-Mel. Este formato es más fácil de procesar para una red neuronal que la onda cruda y ocupa mucho menos espacio.

Básicamente:

- Tomamos el audio crudo de 1 segundo.
- Lo pasamos a un espectrograma de 32 bandas (una matriz de tamaño aproximado 32×101).
- Aplicamos logaritmo para que los valores sean más estables.
- Acomodamos todos los ejemplos para que tengan exactamente el mismo tamaño.

1.2 Arquitectura del autoencoder

Entrenamos un autoencoder, es decir, una red neuronal que:

1. **Comprime** el espectrograma en un vector pequeño (*latent vector*).
2. **Reconstruye** un espectrograma a partir de ese vector.

El modelo tiene dos partes:

- **Encoder:** va achicando el espectrograma de forma progresiva hasta obtener un vector de tamaño z_{dim} (por ejemplo 128, 64, 32, etc.).
- **Decoder:** toma ese vector comprimido y lo vuelve a expandir para recuperar el espectrograma original.

En nuestro caso el encoder reduce todo a un vector de tamaño z_dim mediante capas convolucionales, y el decoder invierte ese proceso usando deconvoluciones.

Finalmente comparamos el espectrograma original con el reconstruido y calculamos la pérdida (MSE).

Esta red nos permite investigar cuál es el mínimo tamaño posible del vector latente sin que el audio reconstruido pierda demasiada calidad.

1.3 Entrenamiento

El modelo se entrenó utilizando:

$$\text{loss} = \text{MSE}(L, \hat{L}), \quad \text{optimizer} = \text{Adam}(lr = 10^{-3}), \quad \text{batch size} = 128.$$

Dado que el objetivo del inciso es explorar cuánto puede comprimirse el audio sin destruir información relevante, probamos cinco tamaños diferentes del vector latente:

$$z_dim \in \{128, 64, 32, 16, 8\}.$$

Para obtener una primera comparación en todas las dimensiones se entrenó cada modelo durante 10 *epochs*. En una segunda etapa, repetimos los experimentos con más *epochs* (40) para observar si era posible mejorar la reconstrucción en los casos más pequeños.

1.4 Análisis de los resultados

La Tabla 1 resume las pérdidas finales obtenidas para cada dimensión con 10 *epochs*:

Dimensión latente	Loss final (MSE)
128	~ 1.90
64	~ 2.21
32	~ 2.76
16	~ 3.57
8	~ 4.79

Table 1: Pérdida final después de 10 *epochs* para cada dimensión del vector latente.

Se observa una tendencia clara, al disminuir la dimensión latente, aumenta la pérdida de reconstrucción.

Esto coincide con lo observado: los audios reconstruidos para $z \leq 16$ comienzan a presentar pérdida de claridad, mientras que $z = 32, 64, 128$ producen reconstrucciones mucho más nítidas.

La Tabla 2 resume las pérdidas finales obtenidas para cada dimensión con 40 *epochs*:

Dimensión latente	Loss final (MSE)
128	~ 1.72
64	~ 2.03
32	~ 2.53
16	~ 3.26
8	~ 4.34

Table 2: Pérdida final después de 40 *epochs* para cada dimensión del vector latente.

Al comparar los resultados de 40 *epochs* con los obtenidos con 10 *epochs*, observamos que todas las dimensiones latentes mejoran su pérdida de reconstrucción. Esto era esperable: con más iteraciones el modelo ajusta mejor sus parámetros y logra una representación más precisa del espectrograma.

Sin embargo, la tendencia general se mantiene: incluso entrenando durante más tiempo, los modelos con dimensiones pequeñas ($z = 8$ y $z = 16$) siguen teniendo pérdidas considerablemente más altas que los de mayor tamaño. La brecha entre dimensiones no se cierra, lo que indica que el problema no era la falta de entrenamiento sino que al comprimir demasiado (8 o 16), el autoencoder ya no puede almacenar suficiente información para reconstruir correctamente el audio.

Por otro lado, $z = 32$ muestra una mejora clara al pasar de 2.76 a aproximadamente 2.53, acercándose más al rendimiento de $z = 64$ y $z = 128$. Esto refuerza la idea de que $z = 32$ es efectivamente la mejor opción ya que logra un nivel de compresión elevado sin perder demasiada calidad.

Además de las métricas de pérdida, en la Figura 1 se ilustra visualmente cómo cambia el espectrograma reconstruido al reducir z_dim . Para $z = 128$ y $z = 64$ se aprecian detalles en el patrón espectral, mientras que a partir de $z = 32$ la imagen comienza a verse más suavizada. Con $z = 16$ y, especialmente, con $z = 8$, se pierde

nitidez en las transiciones y la forma del comando se vuelve más borrosa, lo que coincide con el aumento de la MSE y la peor calidad perceptual observada en la escucha.

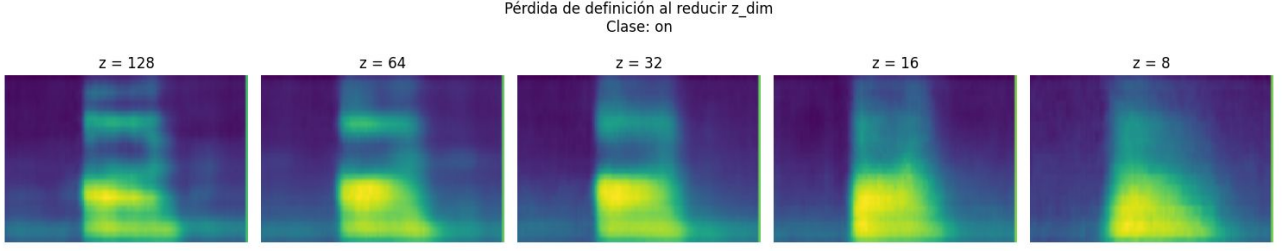


Figure 1: Pérdida de definición al reducir la dimensión latente z_dim para un ejemplo de la clase *on*. De izquierda a derecha: $z_dim \in \{128, 64, 32, 16, 8\}$.

1.5 Conclusión

El autoencoder logra comprimir los audios en diferentes espacios latentes de distinta dimensión, pero presentando un límite inferior práctico. Con los experimentos realizados, concluimos que:

$z_dim = 32$ ofrece el mejor equilibrio entre compresión y calidad.

Si bien $z = 16$ sigue siendo utilizable para algunos audios, la calidad perceptual cae considerablemente, y $z = 8$ resulta insuficiente para una reconstrucción comprensible incluso incrementando el número de *epochs*.

2 Análisis exploratorio del espacio latente

En este ejercicio analizamos la estructura del espacio latente aprendido por el autoencoder. A partir de las conclusiones del Ejercicio 1, tomamos como caso principal $z_dim = 32$ (mejor compromiso entre compresión y calidad) y lo comparamos con una dimensión más pequeña ($z_dim = 16$) y una más grande ($z_dim = 64$). Para cada modelo entrenado se recorrió el conjunto de prueba y se extrajeron los vectores latentes z (salida intermedia del encoder). De esta forma se obtuvo, para cada configuración de z_dim , una matriz $Z \in \mathbb{R}^{N \times z_dim}$ junto con los labels originales de cada audio.

2.1 Reducción de dimensionalidad: PCA y t-SNE

Primero aplicamos PCA con dos componentes para visualizar el espacio latente en 2D. En la Figura 2 se muestran los resultados para $z_dim = 16, 32, 64$, coloreando cada punto según su clase (palabra).

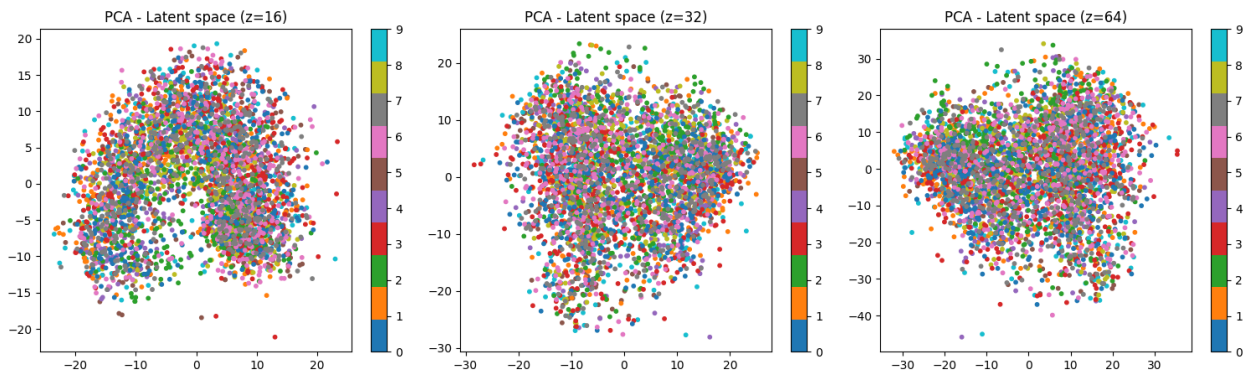


Figure 2: PCA del espacio latente para $z_dim = 16$ (izq.), $z_dim = 32$ (centro) y $z_dim = 64$ (der.).

En los tres casos aparece una nube densa de puntos con fuerte solapamiento entre clases. La proyección lineal a 2D no alcanza para separar los comandos de voz en regiones claramente diferenciadas, lo que sugiere que la información relevante se distribuye en más dimensiones del espacio latente.

Para capturar relaciones no lineales se utilizó luego t-SNE. La Figura 3 muestra los mapas 2D obtenidos también para $z_dim = 16, 32, 64$.

A diferencia de PCA, t-SNE revela grupos más compactos y estructuras más marcadas. Aun así, las clases no se separan de forma perfecta: hay zonas donde distintos colores se superponen. Visualmente se observa que al

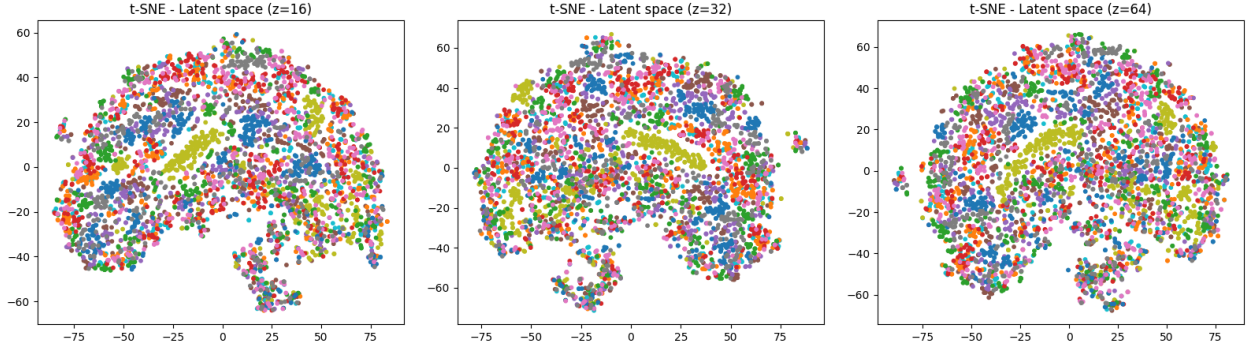


Figure 3: t-SNE del espacio latente para $z_dim = 16$ (izq.), $z_dim = 32$ (centro) y $z_dim = 64$ (der.).

pasar de $z_dim = 16$ a $z_dim = 32$ y $z_dim = 64$ los grupos tienden a ser algo más compactos, lo que indica una mejor capacidad para organizar los audios en el espacio latente cuando la dimensión es mayor.

2.2 Clustering sobre el espacio latente ($z_dim = 32$)

Sobre los vectores latentes con $z_dim = 32$ aplicamos **K-Means** con distintos valores de k . La Figura 4 muestra la curva *elbow* para $k \in [2, 12]$, graficando la inercia (suma de distancias cuadráticas a los centroides).

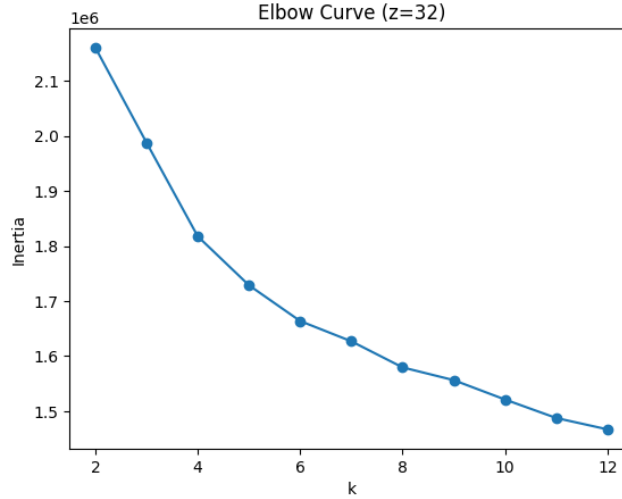


Figure 4: Elbow curve del K-Means sobre el espacio latente con $z_dim = 32$.

La inercia disminuye rápidamente para valores pequeños de k y luego lo hace cada vez más lentamente. No aparece un “codo” muy marcado, aunque el cambio de pendiente alrededor de $k \approx 5$ sugiere una región razonable de compromiso entre número de clusters y calidad del ajuste.

Para cuantificar cuánto se alinean los clusters con las clases reales, calculamos las métricas de **homogeneidad** y **completitud** para $k = 5, 10, 15$. La Tabla 3 resume los valores obtenidos:

k	Homogeneidad	Completitud
5	0.006	0.009
10	0.010	0.010
15	0.086	0.075

Table 3: Métricas de homogeneidad y completitud para K-Means sobre el espacio latente con $z_dim = 32$.

Las métricas se mantienen bajas para todos los valores de k , con una ligera mejora al aumentar la cantidad de clusters. Esto indica que la asignación producida por K-Means no coincide con las diez clases reales del dataset: los clusters contienen mezclas de palabras y cada palabra aparece repartida entre varios grupos.

Este comportamiento es esperable en un autoencoder no supervisado. El modelo fue entrenado exclusivamente para reconstruir espectrogramas, no para separar clases, por lo que el espacio latente refleja similitudes acústicas generales más que categorías de palabras. Además, las propias clases del dataset presentan una alta

superposición espectral (por ejemplo, pares como *on/off*, *yes/no* o *left/right*) lo que dificulta que un método puramente geométrico como K-Means encuentre fronteras claras.

Aumentar la dimensión del espacio latente podría preservar más variabilidad, pero no cambia que estos audios, tal como están representados, no se separan claramente unos de otros. Incluso para z mayores las clases no forman agrupamientos compactos o disjuntos. Del mismo modo, incrementar k produce clusters más pequeños y ligeramente más puros, pero eso no alcanza para descubrir grupos que realmente correspondan a las palabras del dataset.

2.3 Conclusión

En conjunto, el análisis muestra que el espacio latente aprendido por el autoencoder captura patrones útiles, visible en las proyecciones t-SNE donde aparecen ciertas estructuras globales, pero no organiza las muestras en diez grupos bien definidos que coincidan con las clases originales. Al incrementar la dimensión latente se obtiene una representación algo más estable y con clusters ligeramente más compactos, pero las mejoras son marginales más allá de $z_dim = 32$.

3 Denoising con Autoencoder

Este ejercicio analiza la capacidad del autoencoder entrenado en el Ejercicio 1 (con dimensión latente $z_{dim} = 32$) para realizar *denoising* sobre los comandos de voz. El objetivo es observar si el espacio latente aprendido permite reconstruir correctamente la estructura espectral del audio aun cuando es corrompido con distintos niveles de ruido.

Todo el análisis se realiza en el dominio log-mel. Como estamos usando 32 filtros mel, no contamos con una forma fiable de volver al waveform a partir del espectrograma. Igual, esto no afecta el objetivo del ejercicio, ya que lo que queremos evaluar es cómo responde el autoencoder al ruido en la representación mel, no reconstruir el audio exacto.

3.1 Metodología

Para un ejemplo del conjunto de test se aplicó el siguiente procedimiento:

1. Tomar el audio original (*wav*).
2. Generar versiones ruidosas con niveles de ruido $n \in \{0.01, 0.05, 0.1, 0.2\}$:

$$wav_{noisy} = wav + n \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1)$$

3. Convertir cada señal a su espectrograma *log-mel*.
4. Pasar el mel ruidoso por el autoencoder para obtener el mel reconstruido.
5. Comparar visualmente:
 - mel original,
 - mel con ruido,
 - mel reconstruido (denoised).

3.2 Resultados

A continuación se presentan los resultados para los cuatro niveles de ruido. Cada figura contiene, de izquierda a derecha:

Mel original | Mel con ruido | Mel reconstruido

Ruido = 0.01

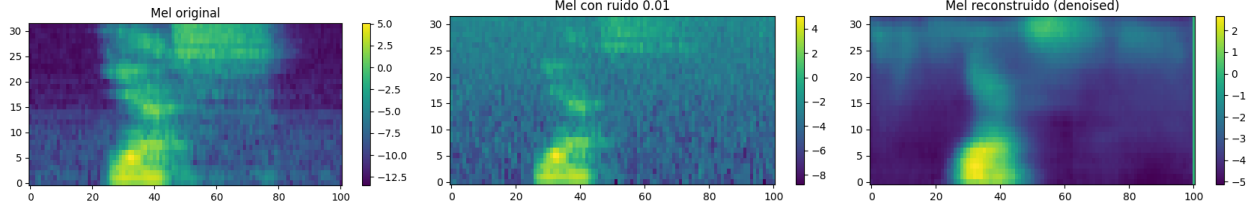


Figure 5: Comparación para nivel de ruido $n = 0.01$: original (izq.), ruidoso (centro) y reconstruido (der.).

Con ruido bajo, la reconstrucción preserva correctamente la envolvente espectral: las regiones de energía se mantienen y el autoencoder filtra la mayor parte del ruido de alta frecuencia.

Ruido = 0.05

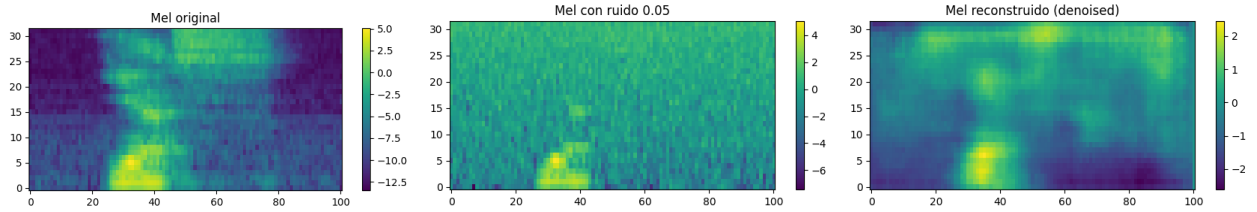


Figure 6: Comparación para nivel de ruido $n = 0.05$: original (izq.), ruidoso (centro) y reconstruido (der.).

El autoencoder continúa suavizando de manera efectiva la señal. A pesar del mayor nivel de ruido, la estructura global se mantiene clara y reconocible.

Ruido = 0.1

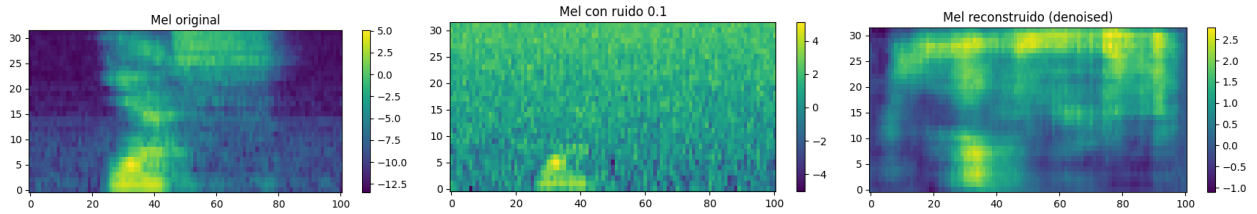


Figure 7: Comparación para nivel de ruido $n = 0.1$: original (izq.), ruidoso (centro) y reconstruido (der.).

Para este nivel moderado de ruido, la reconstrucción sigue siendo buena, aunque comienza a observarse pérdida de detalle en las frecuencias altas.

Ruido = 0.2

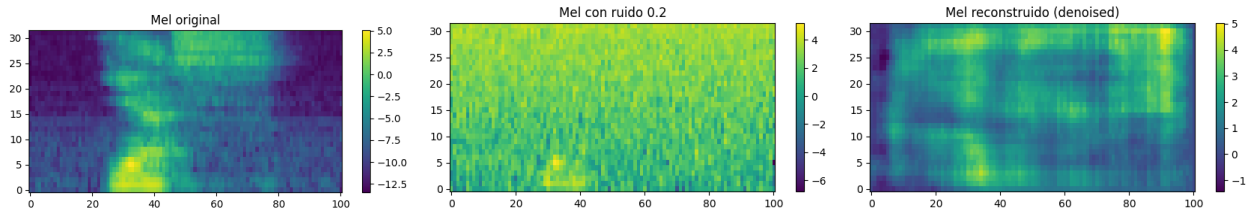


Figure 8: Comparación para nivel de ruido $n = 0.2$: original (izq.), ruidoso (centro) y reconstruido (der.).

Con ruido elevado, el autoencoder recupera parcialmente la estructura principal, pero se observa un exceso de suavizado y pérdida de detalle fino. Aun así, el patrón general del comando se mantiene visible.

3.3 Conclusión

Los resultados muestran que el autoencoder entrenado con $z_{dim} = 32$ posee una capacidad robusta de filtrado en el dominio log-mel. Para niveles bajos y moderados de ruido ($n \leq 0.1$), el modelo logra reconstrucciones limpias, preservando la envolvente espectral y las regiones de mayor energía del comando, aun cuando el ruido distorsiona notablemente la señal original.

Si bien ante ruido muy elevado ($n = 0.2$) se observa pérdida de detalle y un suavizado más pronunciado, el autoencoder mantiene la estructura general del mel, lo que indica que el espacio latente aprendido captura de manera efectiva las características esenciales de la señal.

En conjunto, estos resultados confirman que el modelo no sólo comprime de forma eficiente la representación mel, sino que además es capaz de separar señal y ruido de manera estable, validando la elección de $z_{dim} = 32$ como un tamaño latente apropiado para tareas de compresión y denoising.

3.4 Entrenamiento de un modelo orientado exclusivamente al denoising

El autoencoder utilizado en este ejercicio fue entrenado de manera no supervisada para reconstruir el espectrograma de entrada. Esto significa que el modelo aprende a reproducir cualquier contenido presente en el mel, incluyendo tanto la señal como el ruido, sin distinguir entre ellos. Si el objetivo fuera entrenar un modelo exclusivamente para quitar ruido, sería necesario modificar varios aspectos del entrenamiento.

En primer lugar, el entrenamiento debería ser *supervisado*, utilizando pares $(x_{ruidoso}, x_{limpio})$. Esto permite que el modelo aprenda de manera explícita a transformar una señal degradada en su versión limpia, forzándolo a identificar qué parte de la energía pertenece al ruido y cuál a la voz.

Además, sería conveniente emplear arquitecturas pensadas para restauración de señales que preservan mejor los detalles relevantes mientras atenúan el ruido.

Finalmente, se utilizarían métricas específicas de calidad de audio (como SNR, SDR o PESQ) en lugar de una pérdida MSE pura, que tiende a producir reconstrucciones excesivamente suavizadas.

En síntesis, un modelo diseñado exclusivamente para *denoising* requeriría entrenamiento supervisado, una arquitectura orientada a restauración y métricas de audio adecuadas, lo cual permitiría obtener resultados considerablemente mejores que los de un autoencoder no supervisado.

4 Modificación y combinación de vectores latentes

En este ejercicio exploramos si el espacio latente aprendido por el autoencoder permite generar nuevas variantes de audio aplicando operaciones simples sobre el vector latente z . Trabajamos con el modelo entrenado con $z_{dim} = 32$, ya que en los incisos anteriores mostró el mejor equilibrio entre compresión y calidad de reconstrucción.

Analizamos dos transformaciones:

1. Interpolación entre dos clases ($yes \rightarrow no$).
2. Perturbación del vector latente mediante ruido.

En todos los casos, luego de modificar z se utilizó el decoder del autoencoder para obtener nuevamente un espectrograma log-mel y su audio correspondiente.

4.1 Interpolación entre clases ($yes \rightarrow no$)

Tomamos un ejemplo real de la clase *yes* y uno de la clase *no*, los codificamos a sus vectores latentes z_{yes} y z_{no} y realizamos una interpolación lineal:

$$z(\alpha) = \alpha \cdot z_{yes} + (1 - \alpha) \cdot z_{no}, \quad \alpha \in \{1.0, 0.75, 0.5, 0.25, 0.0\}.$$

Cada vector $z(\alpha)$ se decodificó nuevamente a mel y a waveform.

La Figura 9 muestra los espectrogramas generados.

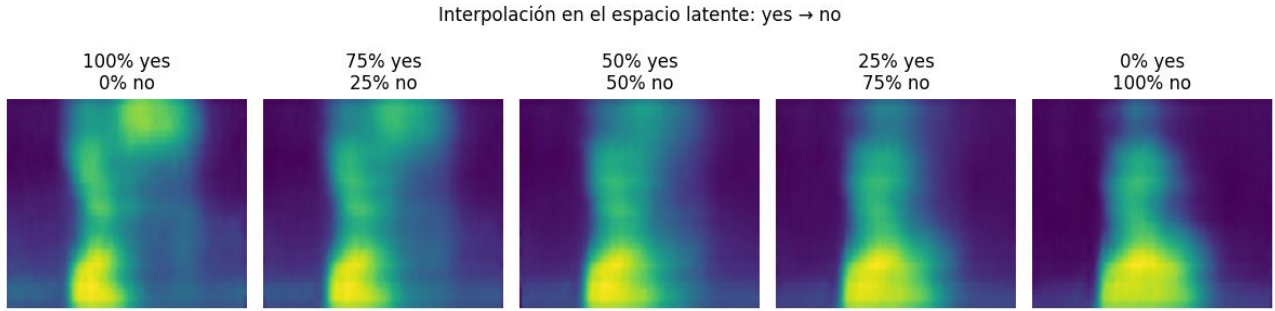


Figure 9: Interpolación en el espacio latente desde *yes* (izq.) hacia *no* (der.) en cinco pasos.

Interpretación

- Para $\alpha = 1.0$ el espectrograma corresponde claramente a un *yes*, mientras que para $\alpha = 0.0$ coincide con un *no*.
- En los valores intermedios aparecen espectrogramas “mixtos”, con patrones que no se corresponden del todo con ninguna de las dos palabras.
- Esto es esperable: la interpolación lineal combina características acústicas de ambas clases, generando sonidos híbridos difíciles de interpretar lingüísticamente.

En resumen, el espacio latente es lo suficientemente regular como para que interpolaciones suaves generen señales coherentes, pero no mantiene una estructura semántica lineal (mezclar *yes* y *no* no produce un comando reconocible).

4.2 Agregando ruido al vector latente

Partiendo del vector z_{yes} , se añadieron perturbaciones de ruido en distintas magnitudes:

$$z_{\text{noisy}} = z_{\text{yes}} + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad \sigma \in \{0.1, 0.5, 1.0, 2.0\}.$$

La Figura 10 muestra el efecto de estas perturbaciones sobre el mel decodificado.

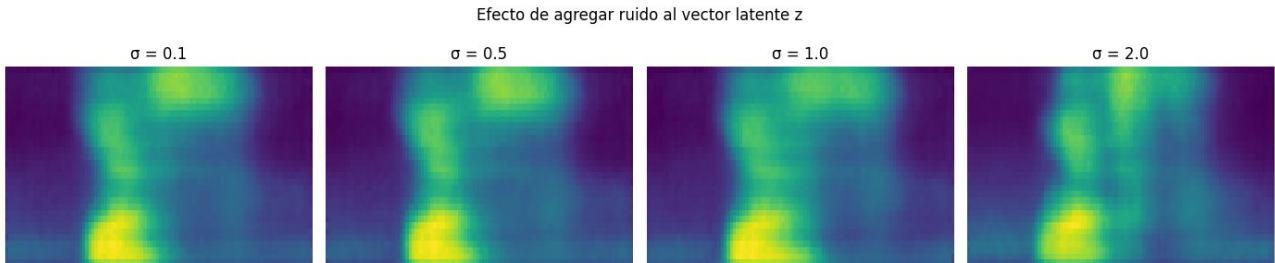


Figure 10: Efecto de agregar ruido al vector latente de la palabra *yes*.

Interpretación

- Para σ pequeños (0.1, 0.5), el mel reconstruido sigue pareciéndose al *yes* original, aunque con ciertas distorsiones.
- Para $\sigma = 1.0$, el patrón acústico ya se deforma lo suficiente como para perder rasgos característicos de la palabra.
- Para $\sigma = 2.0$, la señal generada deja de parecer un comando y pasa a ser un sonido difuso, lo que indica que la perturbación llevó el vector a una región del espacio latente sin estructura lingüística.

En síntesis, el modelo es relativamente estable ante pequeñas perturbaciones, pero el espacio latente no está organizado de forma que sonidos similares queden agrupados muy cerca: moverse demasiado aleja la representación de cualquier comando real.

Conclusión

Los resultados muestran que el espacio latente aprendido por el autoencoder es suave y permite generar variantes continuas de una señal, pero no preserva de forma lineal la semántica de las palabras.

- La interpolación *yes*→*no* produce espectrogramas intermedios válidos acústicamente, aunque no corresponden a una palabra reconocible.
- Agregar ruido pequeño mantiene la estructura general del comando, pero ruidos mayores llevan a regiones sin significado lingüístico.

Es interesante contrastar este resultado con el ejercicio de denoising del punto anterior. Cuando agregamos ruido directamente sobre el waveform, el modelo debe lidiar con dos fuentes simultáneas de distorsión: el ruido agregado y la inevitable degradación producida por el propio proceso de compresión-descompresión del autoencoder. En cambio, al perturbar únicamente el vector latente, el ruido se introduce luego de la compresión, es decir, sobre una representación ya filtrada y más estable. Por esta razón, pequeñas perturbaciones en el espacio latente generan variaciones suaves y controladas, mientras que el ruido añadido en el dominio de la señal original es mucho más difícil de eliminar en la reconstrucción final. Esto muestra que el espacio latente actúa como un filtro natural, pero la calidad del denoising depende críticamente de en qué etapa se introduce la perturbación.

En conjunto, el experimento confirma que el autoencoder captura regularidades acústicas útiles, pero no organiza el espacio latente siguiendo fronteras entre clases. Por lo tanto, es posible manipular vectores latentes para generar nuevos sonidos, pero estas transformaciones no producen transiciones semánticamente interpretables.

Conclusión general

A lo largo del trabajo analizamos cómo un autoencoder puede comprimir y reconstruir comandos de voz a partir de su representación log-Mel. Los experimentos muestran que, aunque todas las configuraciones logran capturar la estructura general de los audios, existe un límite práctico de compresión: a partir de $z_{\text{dim}} = 16$ la calidad de reconstrucción se deteriora claramente, mientras que $z_{\text{dim}} = 32$ ofrece el mejor equilibrio entre tamaño del vector latente y fidelidad perceptual.

El análisis del espacio latente revela que el modelo organiza los ejemplos según similitudes acústicas globales, pero no separa de forma nítida las clases del dataset. Esto es esperable en un entrenamiento no supervisado cuyo objetivo es reconstruir, y no discriminar entre palabras.

En los experimentos de denoising observamos que el autoencoder posee una robustez moderada frente al ruido: es capaz de recuperar parte de la estructura principal del espectrograma aún cuando la señal de entrada está degradada. Sin embargo, al escuchar los audios reconstruidos el ruido no desaparece por completo, ya que se suma la distorsión propia del proceso de compresión y descompresión. Este comportamiento contrasta con la manipulación del vector latente: cuando el ruido se introduce después de la compresión (directamente en z), las perturbaciones producen variaciones más suaves y controladas, lo que muestra que el espacio latente constituye una representación más estable que el waveform original.

Finalmente, las operaciones sobre z (interpolación entre clases y perturbaciones de ruido) evidencian que el espacio latente aprendido es continuo y permite generar señales acústicamente coherentes. No obstante, estas transformaciones no preservan la semántica de manera lineal: los ejemplos intermedios entre *yes* y *no* no corresponden a palabras reconocibles.

En conjunto, los resultados demuestran que los autoencoders son una herramienta efectiva para comprimir y analizar señales de voz, y confirman que $z_{\text{dim}} = 32$ es una elección adecuada para lograr una representación compacta sin comprometer excesivamente la calidad de reconstrucción.

5 Generación de audios mediante muestreo en el espacio latente y GAN (Opcional)

En este ejercicio analizamos si el espacio latente aprendido por el autoencoder permite generar audios completamente nuevos. A diferencia del inciso anterior, donde partíamos de vectores latentes reales, acá intentamos producir vectores latentes “desde cero” y decodificarlos para obtener nuevos espectrogramas y sus audios correspondientes.

5.1 Con autoencoder

Primero recolectamos 5000 vectores latentes reales de dimensión 32, obtenidos pasando audios verdaderos por el encoder del autoencoder entrenado. Con ese conjunto estimamos una distribución estadística sencilla:

$$z \sim \mathcal{N}(\mu, \Sigma),$$

donde:

- μ es la media de todos los vectores latentes reales,
- Σ es la matriz de covarianzas (cómo varía cada dimensión).

Con esa distribución como aproximación del espacio latente, generamos nuevos audios sintéticos mediante:

1. Muestrear un vector latente nuevo:

$$z_{\text{new}} \sim \mathcal{N}(\mu, \Sigma).$$

2. Pasarlo por el decoder del autoencoder, obteniendo un log-mel sintético.
3. Convertir ese log-mel nuevamente a waveform mediante InverseMelScale y Griffin-Lim.

Se generaron diez audios nuevos siguiendo este procedimiento.

Los diez audios generados se reconstruyen sin errores y tienen la duración correcta (≈ 1 segundo). Sin embargo, todos comparten una misma característica: **no se parecen en absoluto a comandos de voz**. Los audios contienen:

- ausencia total de vocales o consonantes,
- señales que suenan a silencio o a ruido muy débil.

Lo mismo se observa en los espectrogramas: no aparecen las bandas de energía típicas de palabras como *yes*, *no* o *stop*. En la Figura 11 se muestran los log-mel generados.

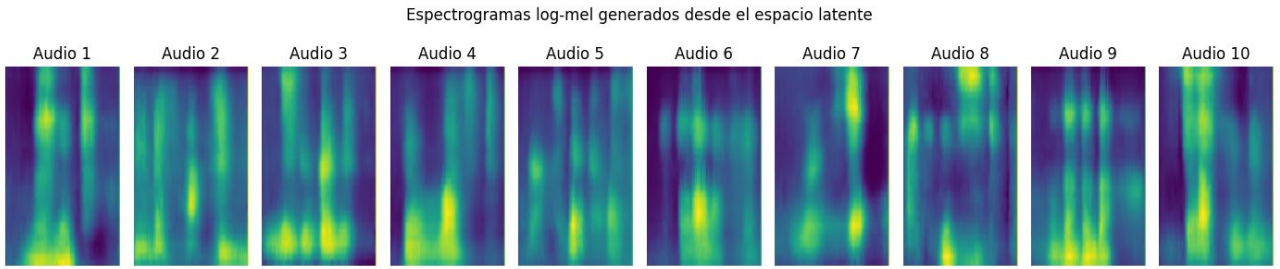


Figure 11: Log-mel espectrogramas generados a partir de vectores latentes muestreados

5.2 Con GAN

Luego, se entrenó una GAN simple trabajando directamente en el espacio latente, utilizando un generador y discriminador. El objetivo fue evaluar si la GAN podía aprender la distribución de los vectores latentes reales y generar latentes más “creíbles” que luego el decoder pudiera transformar en audio.

El entrenamiento se realizó en un principio durante 200 epochs y se registraron las pérdidas del generador y el discriminador, mostradas en la Figura 12.

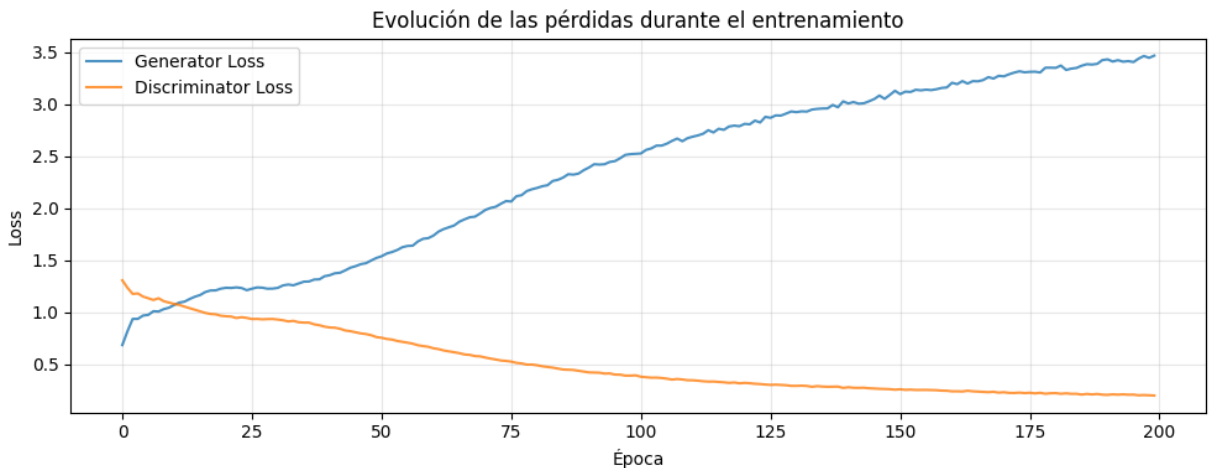


Figure 12: Evolución de las pérdidas durante el entrenamiento de la GAN.

Las curvas muestran que el discriminador reduce su pérdida continuamente, indicando que aprende a distinguir cada vez mejor los vectores reales de los generados. Por el contrario, la pérdida del generador aumenta luego de las primeras 10–15 epochs, señalando que el discriminador comienza a dominar. Visualmente y escuchando los audios generados confirmamos que entrenar más allá de cierto punto no mejoraba la calidad de los espectrogramas; por eso seleccionamos 11 epochs como un equilibrio razonable, ya que es el punto donde tanto la loss del generador y el discriminador se encuentran más balanceadas.

También se analizaron los espectrogramas generados por la GAN a las 10, 20 y 30 epochs.

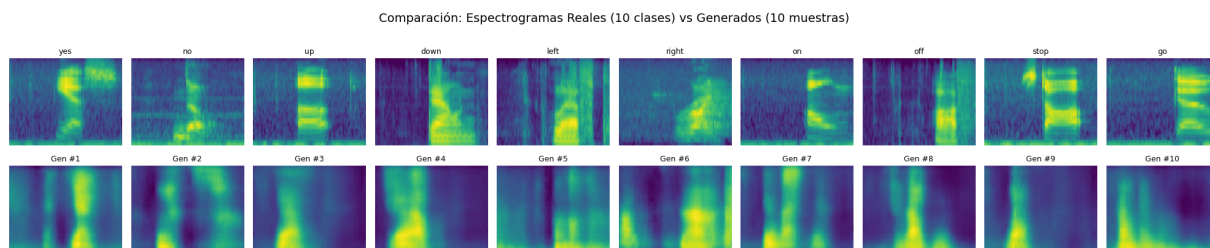


Figure 13: Espectrogramas generados con 10 epochs de entrenamiento.

Con 10 epochs aparecen patrones suaves y zonas de energía diferenciadas, aunque sin formar estructuras lingüísticas reales. Aun así, es la etapa donde la GAN produce sus muestras más razonables: borrosas, pero estables y no completamente colapsadas.

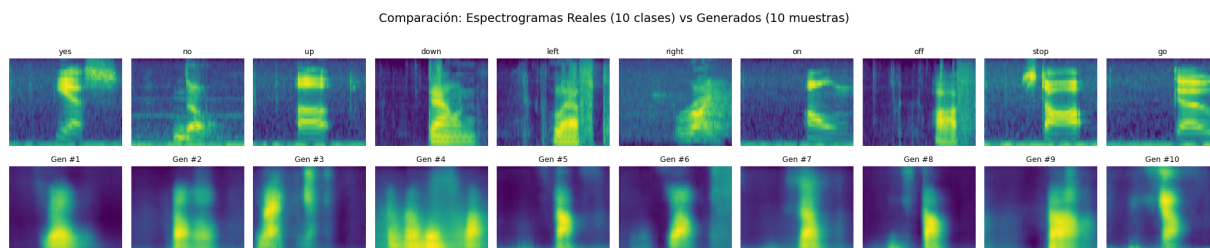


Figure 14: Espectrogramas generados con 20 epochs de entrenamiento.

A las 20 epochs los espectrogramas pierden un poco de contraste y se vuelven más homogéneos que antes. El generador comienza a repetir patrones ciertos patrones. Más entrenamiento no implica mejor calidad en todos los casos, algunos audios parecían contener palabras pero menos que con 10 epochs.

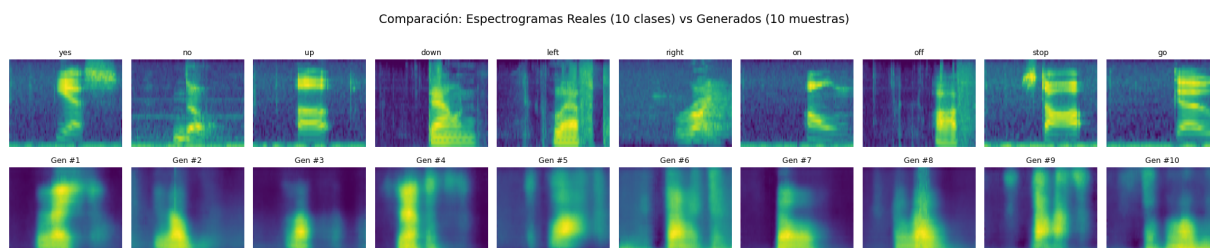


Figure 15: Espectrogramas generados con 30 epochs de entrenamiento.

Con 30 epochs los espectrogramas se aún más borrosos, con formas poco estructuradas. La información es casi ruido suavizado y no corresponde a ningún comando de voz.

5.3 Discusión

Aunque el autoencoder reconstruye bien audios reales, no sirve para generar audios nuevos. Consideramos que las razones principales son que no fue entrenado para crear, sino para copiar entradas y que el espacio latente útil es pequeño lo que produce ruido.

Por otro lado, la GAN aporta una aproximación distinta al intentar aprender la distribución latente, pero su arquitectura simple (sin convoluciones ni estructura temporal) limita fuertemente la calidad de lo generado. Además, la inestabilidad típica de las GAN hace que, después de pocas epochs, el generador colapse y produzca muestras cada vez más borrosas.

La comparación entre 10, 20 y 30 epochs confirma que entrenar más no siempre mejora la calidad: la GAN genera sus mejores (aunque aún insuficientes) resultados alrededor de las 10 epochs.

5.4 Conclusión

Tanto el autoencoder como la GAN permitieron explorar distintas formas de generar audios sintéticos a partir del espacio latente. Si bien ninguno de los dos modelos logró producir comandos de voz claramente reconocibles, el autoencoder pudo reconstruir correctamente la duración y la estructura global del audio, y la GAN logró imitar ciertos patrones energéticos del espacio latente, logrando que algunas palabras pudieran reconocerse en ciertos audios.

Los resultados sugieren que las arquitecturas utilizadas no son lo suficientemente expresivas para capturar toda la variabilidad acústica del dataset. Aun así, el análisis de las pérdidas y de los espectrogramas generados muestra que existe una región de entrenamiento (alrededor de las 10 epochs) donde la GAN produce las muestras más coherentes, lo cual indica que el modelo sí está aprendiendo parte de la distribución latente.

En resumen, los experimentos muestran que es posible generar representaciones latentes “razonables”, pero que para obtener audios realistas se necesitarían modelos más avanzados. Otras alternativas más complejas podrían capturar mejor la estructura temporal y espectral del audio, permitiendo avanzar hacia una generación más natural y con contenido lingüístico reconocible.